

AstraNotes: Development Environment and First Realization Slices

CSEN 296B-2 Week 6 Lab | Instructor: Paddu Melanahalli

1. Development Direction

Language: Python 3.11+.

App type: Web application using Flask with Jinja2 templates.

Storage: SQLite for the development setup. The schema includes a placeholder user_id column so that user accounts can be added later without restructuring the tables.

Encryption (next slice): the cryptography library, with the symmetric key held in a configurable key store.

I went with a web direction rather than desktop because that is the suggested baseline for AstraNotes this quarter. Python is the language I am most productive in for application work, sqlite3 is in the standard library, and Flask stays out of the way for the slice work this lab focuses on. Heavier frameworks would have added more tooling than the create-note and list-notes slices need at this stage.

The structural classes from my UML design (Note, SecureNote, NoteService, NoteRepository, LocalStorage, UserSession, VersionHistory, SearchIndex) carry over to a web architecture without changes. The deployment view will need updating to reflect a server-backed runtime, which I will fold into the next refinement of the design package.

2. Project Structure and Run Environment

The repository mirrors the module boundaries from my class diagram. Each top-level package corresponds to a layer in the design, and nothing crosses layers in the wrong direction. The web routes never import storage directly, and the domain never imports services.

```
astranotes/
|-- src/astranotes/
|   |-- domain/
|   |   |-- note.py          # Note, SecureNote      (domain layer)
|   |-- services/
|   |   |-- note_service.py  # NoteService        (service layer)
|   |-- storage/
|   |   |-- local_storage.py # LocalStorage      (SQLite backend)
|   |   |-- note_repository.py # NoteRepository    (domain-shaped I/O)
|   |-- web/
|   |   |-- app.py           # Flask factory + routes
|   |   |-- templates/       # Jinja2 templates (base, notes, placeholder)
|   |   |-- static/style.css  # minimal CSS
|-- tests/
|   |-- test_slices.py        # pytest unit tests for the two slices
|-- docs/
|   |-- architecture notes; UML package lives here
|-- pyproject.toml           # build, run, and test configuration
|-- README.md
```

Build, run, and test. `pip install -e ".[dev]"` installs the package in editable mode. `astranotes` starts the Flask development server. `pytest` runs the slice tests. All 6 currently pass.

3. UI Shell

The shell is a Flask app with Jinja2 templates. A base template supplies the navigation bar and a flash-message area, and child templates render the actual page content. The main notes page is a two-column layout with a notes-list sidebar on the left and a new-note form on the right.

- **Navigation bar:** three links for Notes, Profile, and Settings. Profile and Settings have real routes that render placeholder pages, so the navigation structure is in place and will not need to be restructured when those slices get filled in later.
- **Sidebar (notes list):** populated by `NoteService.list_notes()`. This is the Slice 2 surface.
- **Workspace (right column):** a new-note form that posts to the create route. The open-into-editor flow is the next slice after this lab.
- **Flash area:** handles two categories of feedback. Input validation errors render as a warning style. Storage failures render as a danger style with a different color. The two are visually distinct so the user can tell what kind of failure occurred.

The Flask routes themselves carry almost no logic of their own. They parse the request, call `NoteService`, and render a template. Anything more complicated than that belongs in a service, not in a route.

4. First Functionality Slices

Slice 1: Create Note (realizes FR-1)

Three pieces collaborate on this slice. The `Note` domain class enforces the title invariant in its constructor. `NoteService.create_note` coordinates the workflow. `NoteRepository.save` persists through SQLite. Putting validation on the constructor rather than in the service means no `Note` object can exist in an invalid state, regardless of who creates it.

```
# src/astranotes/domain/note.py
@dataclass
class Note:
    title: str
    content: str
    note_id: str = field(default_factory=lambda: str(uuid4()))
    created_at: datetime = field(default_factory=_now)
    last_modified: datetime = field(default_factory=_now)

    def __post_init__(self) -> None:
        # Domain invariant: title must be a non-empty string after trim.
        if not isinstance(self.title, str) or not self.title.strip():
            raise ValueError("Note title must be a non-empty string.")
        if not isinstance(self.content, str):
            raise ValueError("Note content must be a string.")
        self.title = self.title.strip()

# src/astranotes/services/note_service.py
class NoteService:
    def __init__(self, repository: NoteRepository) -> None:
        self._repo = repository

    def create_note(self, title: str, content: str = "") -> Note:
        """Create-Note workflow (FR-1). Validates input, builds the
        domain object, persists it. The Note constructor enforces the
        title invariant, so the service does not re-validate."""
        note = Note(title=title, content=content)
        self._repo.save(note)
        return note
```

Failure handling. `LocalStorage` catches `sqlite3.Error` and re-raises it as a domain-level `StorageError`, so the route handlers never have to know SQLite exists. The route catches `ValueError` (bad input) and `StorageError` (write failure) as two separate paths and flashes a different message style for each. This addresses the graceful-failure non-functional requirement that my traceability matrix had flagged as weakly supported.

Slice 2: List Notes

`NoteService.list_notes()` calls `NoteRepository.load_all()`, which returns every note ordered by `last_modified DESC`. This is the read side that every later slice (edit, delete, search) needs to exist before they make sense. The row-to-Note conversion lives in the repository so the rest of the system never sees SQL rows.

```
# src/astranotes/storage/note_repository.py
def load_all(self) -> list[Note]:
    """Return every note, most-recently-modified first."""
    rows = self._storage.query(
        "SELECT * FROM notes ORDER BY last_modified DESC"
    )
    return [self._row_to_note(r) for r in rows]
```

Tests. Six pytest cases cover both slices, using a `tmp_path` fixture so the SQLite file is throwaway per test: create-and-persist round trip, empty-title rejection, whitespace trimming, list ordering, the empty-store case, and a storage-corruption case added during the audit in Section 6. All six pass.

5. Traceability Note

Mapping from this lab's slices back to the requirements they realize:

Slice	Realizes (Req IDs)	UML Elements Realized	Status After This Lab
Slice 1: Create Note	FR-1 (Create note); NFR-2 (Graceful failure)	Note class; NoteService.createNote(); NoteRepository.save(); LocalStorage; create-and-save activity flow	Code path matches design. Happy path and invalid-input failure path covered by tests.
Slice 2: List Notes	Supports FR-2 (Edit) and FR-3 (Delete) by exposing notes in the UI	NoteRepository.loadAll(); NoteService (list workflow); Note (read-side)	Notes load from SQLite ordered by lastModified DESC. Empty-store case handled.

Two specific status improvements this lab earns against the traceability matrix I built earlier. **FR-1** moves from "Fully Traced in design" to "Fully Traced and realized in code with passing tests." The **graceful-failure non-functional requirement** (NFR-2) was weakly traced because only the create-flow validation loop existed in the design. There are now two real error paths in running code (`ValueError` for invalid input, `StorageError` for write failures), each with its own UI response.

6. Quality Audit on Slice 1

I went back and reviewed the create-note slice for readability, responsibility clarity, and failure handling. Two findings were worth fixing.

Finding 1: Storage abstraction was leaking

`LocalStorage.query()` was returning `list[sqlite3.Row]`. The whole point of `LocalStorage` in my class diagram is to be the only place that knows about SQLite. But if the return type is a SQLite-specific type, then `NoteRepository._row_to_note` depends on `sqlite3`'s row interface to do its job. The boundary I claimed in the design did not actually exist in the code. Swapping SQLite for a different driver later would have silently broken the repository.

Fix. Converted rows to plain dicts inside `LocalStorage.query()`.

```
# before
def query(self, sql: str, params: tuple = ()) -> list[sqlite3.Row]:
```

```

        with self._connect() as conn:
            return list(conn.execute(sql, params).fetchall())

# after
def query(self, sql: str, params: tuple = ()) -> list[dict]:
    with self._connect() as conn:
        rows = conn.execute(sql, params).fetchall()
        # Convert to plain dicts so callers never see sqlite3.Row.
        return [dict(r) for r in rows]

```

NoteRepository._row_to_note did not need to change because row["title"] works on both sqlite3.Row and dict. The repository now depends on a shape (a dict-like row), not on a driver type.

Finding 2: Claimed failure handling was untested

I had told myself that the StorageError path was covered because the code structure looked right: try/except around sqlite calls, a custom exception type, a route handler that caught it. But none of the original five tests actually exercised the path. I was trusting how the code read instead of what the tests proved.

Fix. Added a test that initializes a valid SQLite store, overwrites the file bytes with garbage, and asserts that the next query raises StorageError.

```

def test_storage_raises_storage_error_on_corrupt_db(tmp_path):
    db_path = tmp_path / "notes.db"
    storage = LocalStorage(db_path)
    storage.execute(
        "INSERT INTO notes(note_id, title, content, "
        "created_at, last_modified) VALUES (?, ?, ?, ?, ?)",
        ("x", "t", "", "2026-01-01", "2026-01-01"),
    )
    # Corrupt the file so the next read fails inside sqlite3.
    db_path.write_bytes(b"this is not a sqlite database file")
    with pytest.raises(StorageError):
        storage.query("SELECT * FROM notes")

```

The test passes, which means the error path is real and not just structural decoration. Test count is now 6.

Debugging and quality lesson

Both findings come from the same mistake. I assumed that because the code looked correct, a property held. Finding 1 was a type that claimed encapsulation but did not provide it. Finding 2 was a code path that looked covered but had no test for it. The lesson is to write the failing test first for any property I want to claim, whether that is a boundary or an error path. If I cannot write a test that would have failed before the code existed, then I do not actually know whether the code does what I think it does.

7. How Copilot Helped and What I Changed

I used GitHub Copilot for the scaffolding work: the pyproject layout, the SQLite schema string, the Flask factory boilerplate, and the first version of the test file. That kind of pattern-heavy code is where Copilot is genuinely faster than typing it myself, and the suggestions matched the standard idioms closely enough that I kept most of them.

Where I overrode the first pass:

Where validation lives. Copilot's first suggestion put the empty-title check inside NoteService.create_note. I moved it onto Note.__post_init__ instead. The reason is simple. If validation lives in the service, a different caller can construct an invalid Note directly. If validation lives in the constructor, an invalid Note cannot exist in the system at all. That is the responsibility-clarity point my class diagram already implies, so making the code match it costs nothing now and avoids a refactor later.

Hiding SQLite from the rest of the system. The first scaffolding Copilot generated had `sqlite3` imports in multiple files and let `sqlite3.Error` propagate all the way to the route handlers. I rewrote that so SQLite lives only inside `LocalStorage`, and storage failures are re-raised as a domain-level `StorageError`. Swapping SQLite out later becomes a change in one file. The audit in Section 6 found that this boundary was still partially broken (the return type was leaking) and tightened it.

Note and VersionHistory. Copilot offered a `versions: list[Version]` attribute on `Note`. I rejected it. My class diagram aggregates `VersionHistory` under `NoteRepository`, not `Note`. Versions should outlive their notes by design. Accepting Copilot's suggestion would have silently changed an architectural decision I had already made.

Alignment with the existing design. Copilot did not have access to my UML class diagram or my traceability matrix, and treated each suggestion as a fresh start. So for each suggestion I had to ask whether it preserved the class names, the layer boundaries, and the responsibility split I had already committed to. Copilot was fast at producing code that looked plausible. It was slower at producing code that matched the design.